

老子软考，扫码下载



老子
软考

老子软考
软考用老子

软件设计师-2017年上半年-选择题

第1题：CPU执行算术运算或者逻辑运算时，常将源操作数和结果暂存在（ ）中。

- A. 程序计数器 (PC)
- B. 累加器 (AC)
- C. 指令寄存器 (IR)
- D. 地址寄存器 (AR)

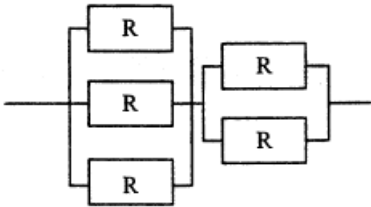
第2题：要判断字长为 16 位的整数 a 的低四位是否全为 0，则（ ）

- A. 将 a 与 0x000F 进行"逻辑与"运算，然后判断运算结果是否等于 0
- B. 将 a 与 0x000F 进行"逻辑或"运算，然后判断运算结果是否等于 F
- C. 将 a 与 0x000F 进行"逻辑异或"运算，然后判断运算结果是否等于 0
- D. 将 a 与 0x000F 进行"逻辑与"运算，然后判断运算结果是否等于 F

第3题：计算机系统中常用的输入/输出控制方式有无条件传送、中断、程序查询和 DMA 方式等。当采用（ ）方式时，不需要 CPU 执行程序指令来传送数据。

- A. 中断
- B. 程序查询
- C. 无条件传送
- D. DMA

第4题：某系统由下图所示的冗余部件构成。若每个部件的千小时可靠度都为 R，则该系统的千小时可靠度为（ ）。



- A. $(1-R^3)(1-R^2)$
- B. $(1-(1-R)^3)(1-(1-R)^2)$
- C. $(1-R^3)+(1-R^2)$
- D. $(1-(1-R)^3)+(1-(1-R)^2)$

第5题：已知数据信息为 16 位，最少应附加（ ）位校验位，才能实现海明码纠错。

- A. 3
- B. 4
- C. 5
- D. 6

第6题：以下关于Cache (高速缓冲存储器)的叙述中，不正确的是（ ）

- A. Cache 的设置扩大了主存的容量
- B. Cache 的内容是主存部分内容的拷贝
- C. Cache 的命中率并不随其容量增大线性地提高
- D. Cache 位于主存与 CPU 之间

第7题：HTTPS 使用（ ）协议对报文进行封装

- A. SSH
- B. SSL
- C. SHA-1
- D. SET

第8题：以下加密算法中适合对大量的明文消息进行加密传输的是（ ）

- A. RSA
- B. SHA-1
- C. MD5
- D. RC5

第9题：假定用户 A、B 分别在 I1 和 I2 两个 CA 处取得了各自的证书，下面（ ）是 A、B 互信的必要条件。

- A. A、B 互换私钥
- B. A、B 互换公钥
- C. I1、I2 互换私钥

D. I1、I2互换公钥

第10题：甲软件公司受乙企业委托安排公司软件设计师开发了信息系统管理软件，由于在委托开发合同中未对软件著作权归属作出明确的约定，所以该信息系统管理软件的著作权由()享有。

- A. 甲
- B. 乙
- C. 甲与乙共同
- D. 软件设计师

第11题：根据我国商标法，下列商品中必须使用注册商标的是()。

- A. 医疗仪器
- B. 墙壁涂料
- C. 无糖食品
- D. 烟草制品

第12题：甲、乙两人在同一天就同样的发明创造提交了专利申请，专利局将分别向各申请人通报有关情况，并提出多种可能采用的解决办法。下列说法中，不可能采用()

- A. 甲、乙作为共同申请人
- B. 甲或乙一方放弃权利并从另一方得到适当的补偿
- C. 甲、乙都不授予专利权
- D. 甲、乙都授予专利权

第13题：数字语音的采样频率定义为 8kHz，这是因为()

- A. 语音信号定义的频率最高值为4kHz
- B. 语音信号定义的频率最高值为8kHz
- C. 数字语音传输线路的带宽只有8kHz
- D. 一般声卡的采样频率最高为每秒8k次

第14题：使用图像扫描仪以300DPI的分辨率扫描一幅3×4英寸的图片，可以得到()像素的数字图像。

- A. 300×300
- B. 300×400
- C. 900×4
- D. 900×1200

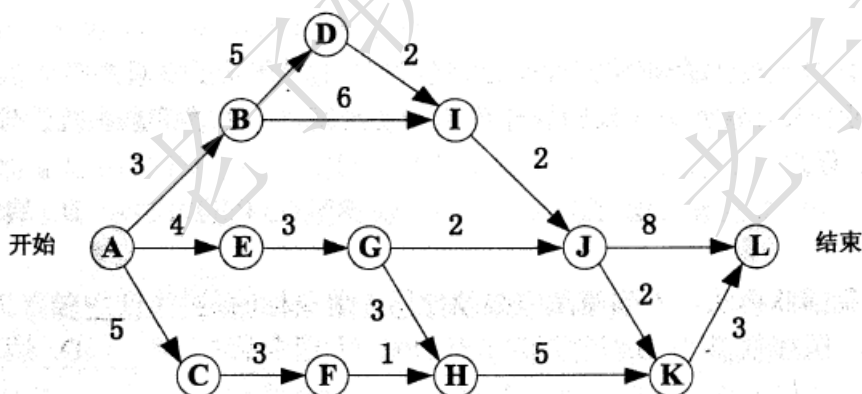
第15题：在采用结构化开发方法进行软件开发时，设计阶段接口设计主要依据需求分析阶段的()。接口设计的任务主要是()。

- A. 数据流程图
- B. E-R图
- C. 状态-迁移图
- D. 加工规格说明

第16题：在采用结构化开发方法进行软件开发时，设计阶段接口设计主要依据需求分析阶段的()。接口设计的任务主要是()。

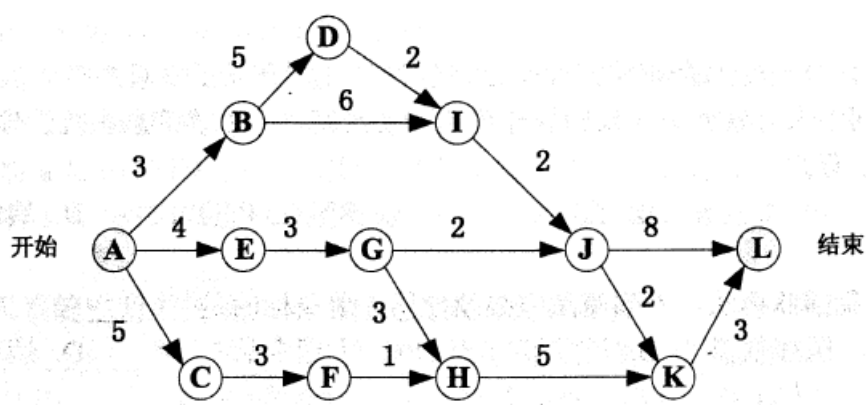
- A. 定义软件的主要结构元素及其之间的关系
- B. 确定软件涉及的文件系统的结构及数据库的表结构
- C. 描述软件与外部环境之间的交互关系，软件内模块之间的调用关系
- D. 确定软件各个模块内部的算法和数据结构

第17题：某软件项目的活动图如下图所示，其中顶点表示项目里程碑，连接顶点的边表示包含的活动，边上的数字表示活动的持续时间(天)，则完成该项目的最少时间为()天。活动BD和HK最早可以从第()天开始。(活动AB、AE和AC最早从第1天开始)



- A. 17
- B. 18
- C. 19

第18题：某软件项目的活动图如下图所示，其中顶点表示项目里程碑，连接顶点的边表示包含的活动，边上的数字表示活动的持续时间(天)，则完成该项目的最少时间为()天。活动BD和HK最早可以从第()天开始。(活动AB、AE和AC最早从第1天开始)



- A. 3和10
- B. 4和11
- C. 3和9
- D. 4和10

第19题：在进行软件开发时，采用无程序员的开发小组，成员之间相互平等;而主程序员负责制的开发小组，由一个主程序员和若干成员组成，成员之间没有沟通。在一个由8名开发人员构成的小组中，无主程序员组和主程序员组的沟通路径分别是()。

- A. 32和8
- B. 32和7
- C. 28和8
- D. 28和7

第20题：在高级语言源程序中，常需要用户定义的标识符为程序中的对象命名，常见的命名对象有()

- ①关键字 (或保留字) ②变量 ③函数 ④数据类型 ⑤注释
- A. ①②③
 - B. ②③④
 - C. ①③⑤
 - D. ②④⑤

第21题：在仅由字符a、b构成的所有字符串中，其中以b结尾的字符串集合可用正规式表示为()

- A. (b|ab)*b
- B. (ab*)*b
- C. a*b*b
- D. (a|b)*b

第22题：在以阶段划分的编译过程中，判断程序语句的形式是否正确属于()阶段的工作。

- A. 词法分析
- B. 语法分析
- C. 语义分析
- D. 代码生成

第23题：某文件管理系统在磁盘上建立了位示图(bitmap)，记录磁盘的使用情况。若计算机系统的字长为 32 位，磁盘的容量为 300GB，物理块的大小为4MB，那么位示图的大小需要()个字。

- A. 1200
- B. 2400
- C. 6400
- D. 9600

第24题：某系统中有3个并发进程竞争资源R，每个进程都需要5个R，那么至少有()个R，才能保证系统不会发生死锁。

- A. 12
- B. 13
- C. 14
- D. 15

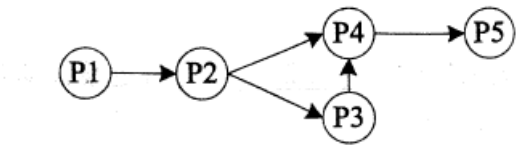
第25题：某计算机系统页面大小为4K，进程的页面变换表如下所示。若进程的逻辑地址为2D16H。该地址经过变换后，其物

理地址应为()

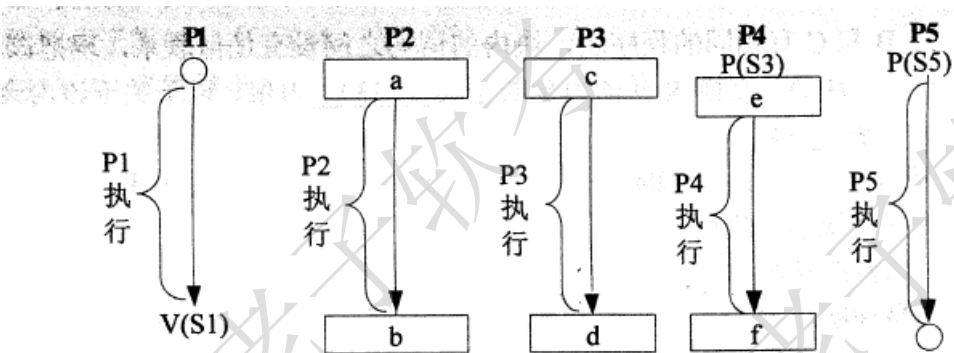
页号	物理块号
0	1
1	3
2	4
3	6

- A. 2048H
- B. 4096H
- C. 4D16H
- D. 6D16H

第26题：进程P1、P2、P3、P4 和P5的前趋图如下所示：

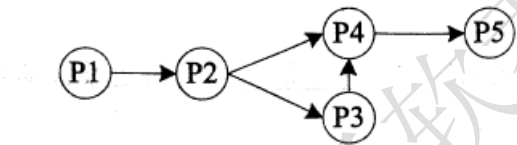


若用PV操作控制进程P1、P2、P3、P4和P5并发执行的过程，需要设置5个信号量S1、S2、S3、S4和S5，且信号量S1~S5的初值都等于零。如下的进程执行图中a和b处应分别填写(26)；c和d处应分别填写(27)；e和f处应分别填写(28)。

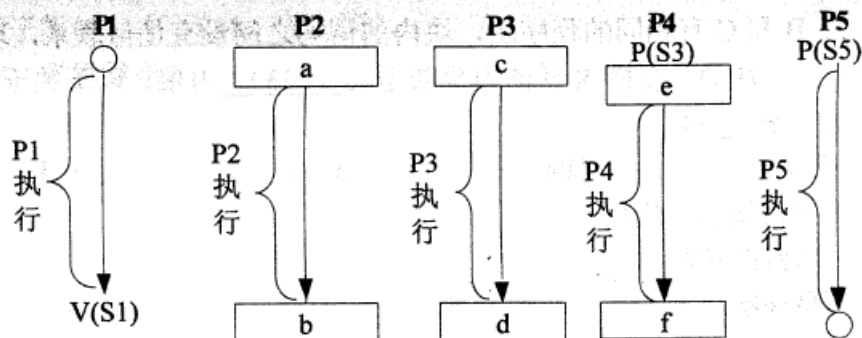


- A. V(S1)和P(S2)V(S3)
- B. P(S1)和V(S2)V(S3)
- C. V(S1)和V(S2)V(S3)
- D. P(S1)和P(S2)V(S3)

第27题：进程P1、P2、P3、P4 和P5的前趋图如下所示：

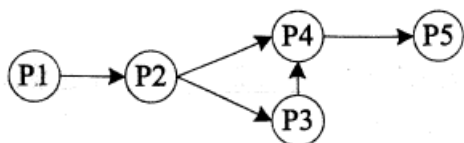


若用PV操作控制进程P1、P2、P3、P4和P5并发执行的过程，需要设置5个信号量S1、S2、S3、S4和S5，且信号量S1~S5的初值都等于零。如下的进程执行图中a和b处应分别填写(26)；c和d处应分别填写(27)；e和f处应分别填写(28)。

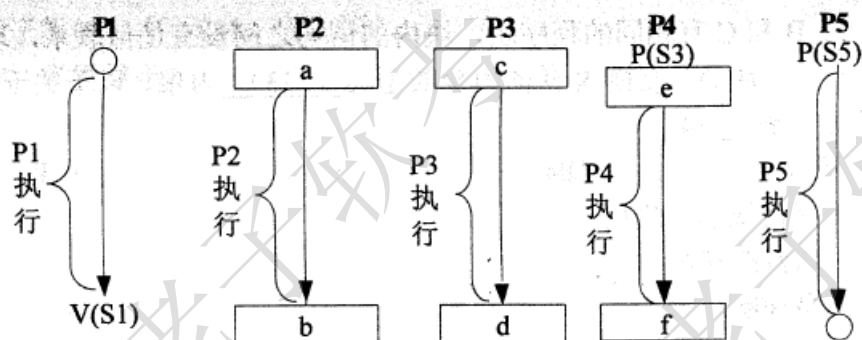


- A. P(S2)和P(S4)
- B. V(S2)和P(S4)
- C. P(S2)和V(S4)
- D. V(S2)和V(S4)

第28题：进程P1、P2、P3、P4和P5的前趋图如下所示：



若用PV操作控制进程P1、P2、P3、P4和P5并发执行的过程，需要设置5个信号量S1、S2、S3、S4和S5，且信号量S1~S5的初值都等于零。如下的进程执行图中a和b处应分别填写(26)；c和d处应分别填写(27)；e和f处应分别填写(28)。



- A. P(S4)和V(S5)
- B. V(S5)和P(S4)
- C. V(S4)和P(S5)
- D. V(S4)和V(S5)

第29题：以下关于螺旋模型的叙述中，不正确的是()

- A. 它是风险驱动的，要求开发人员必须具有丰富的风险评估知识和经验
- B. 它可以降低过多测试或测试不足带来的风险
- C. 它包含维护周期，因此维护和开发之间没有本质区别
- D. 它不适用于大型软件开发

第30题：以下关于极限编程(XP)中结对编程的叙述中，不正确的是()。

- A. 支持共同代码拥有和共同对系统负责
- B. 承担了非正式的代码审查过程
- C. 代码质量更高
- D. 编码速度更快

第31题：以下关于C/S(客户机/服务器)体系结构的优点的叙述中，不正确的是()。

- A. 允许合理地划分三层的功能，使之在逻辑上保持相对独立性
- B. 允许各层灵活地选用平台和软件
- C. 各层可以选择不同的开发语言进行并行开发
- D. 系统安装、修改和维护均只在服务器端进行

第32题：在设计软件的模块结构时，()不能改进设计质量。

- A. 尽量减少高扇出结构
- B. 尽量减少高扇入结构
- C. 将具有相似功能的模块合并

D. 完善模块的功能

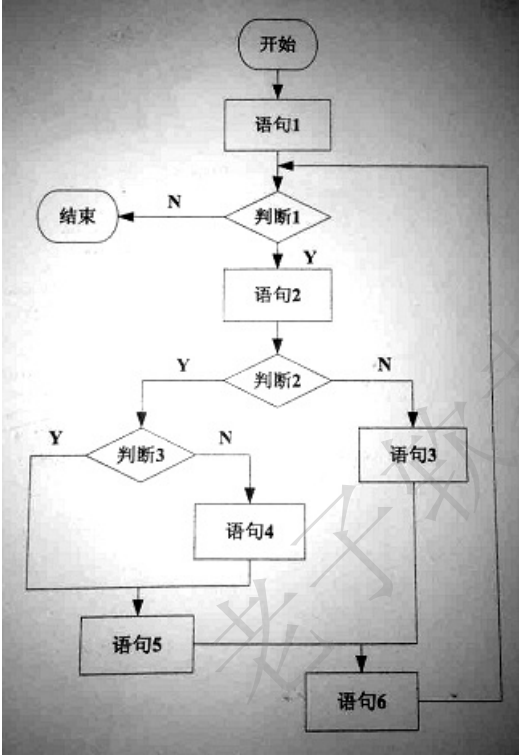
第33题：模块A、B和C有相同的程序块，块内的语句之间没有任何联系，现把该程序块取出来，形成新的模块D，则模块D的内聚类型为()内聚。以下关于该内聚类型的叙述中，不正确的是()。

- A. 巧合
- B. 逻辑
- C. 时间
- D. 过程

第34题：模块A、B和C有相同的程序块，块内的语句之间没有任何联系，现把该程序块取出来，形成新的模块D，则模块D的内聚类型为()内聚。以下关于该内聚类型的叙述中，不正确的是()。

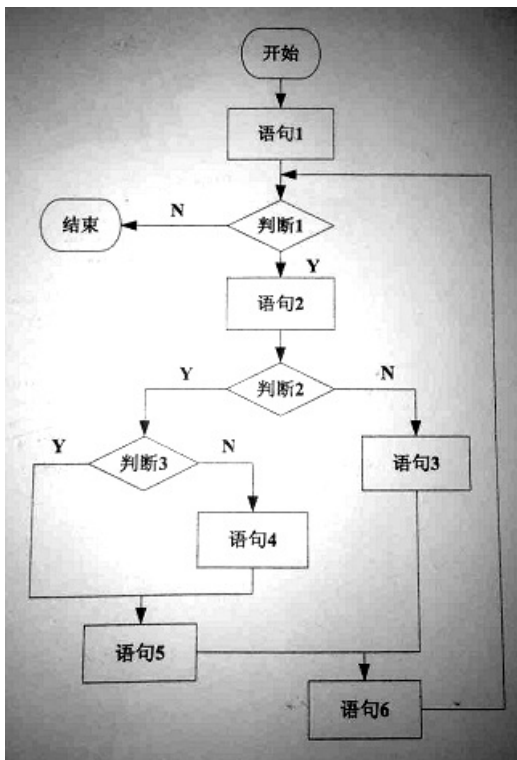
- A. 具有最低的内聚性
- B. 不易修改和维护
- C. 不易理解
- D. 不影响模块间的耦合关系

第35题：对下图所示的程序流程图进行语句覆盖测试和路径覆盖测试，至少需要(35)个测试用例。采用McCabe 度量法计算其环路复杂度为(36)。



- A. 2和3
- B. 2和4
- C. 2和5
- D. 2和6

第36题：对下图所示的程序流程图进行语句覆盖测试和路径覆盖测试，至少需要(35)个测试用例。采用McCabe 度量法计算其环路复杂度为(36)。



- A. 1
- B. 2
- C. 3
- D. 4

第37题：在面向对象方法中，两个及以上的类作为一个类的父类时，称为()，使用它可能造成子类中存在()的成员。

- A. 多重继承
- B. 多态
- C. 封装
- D. 层次继承

第38题：在面向对象方法中，两个及以上的类作为一个类的父类时，称为()，使用它可能造成子类中存在()的成员。

- A. 动态
- B. 私有
- C. 公共
- D. 二义性

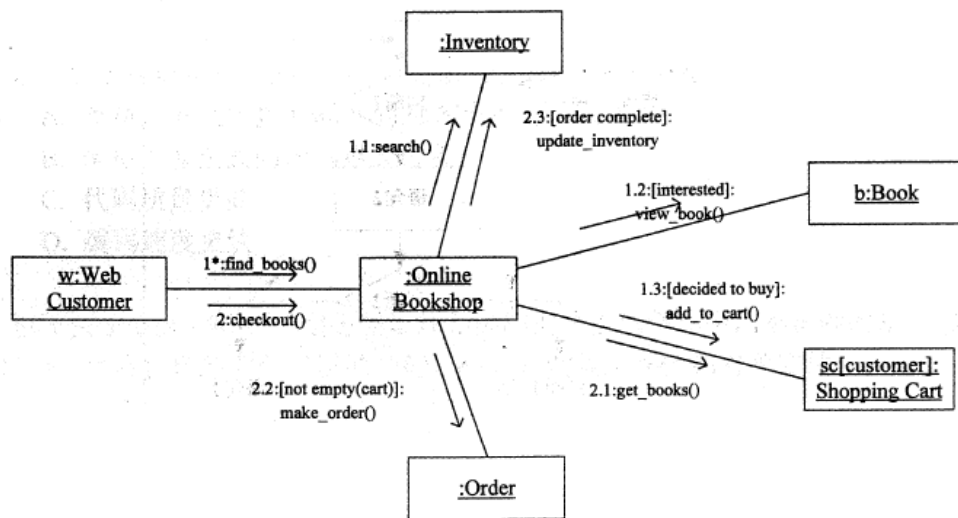
第39题：采用面向对象方法进行软件开发，在分析阶段，架构师主要关注系统的()。

- A. 技术
- B. 部署
- C. 实现
- D. 行为

第40题：在面向对象方法中,多态指的是()

- A. 客户类无需知道所调用方法的特定子类的实现
- B. 对象动态地修改类
- C. 一个对象对应多张数据库表
- D. 子类只能够覆盖父类中非抽象的方法

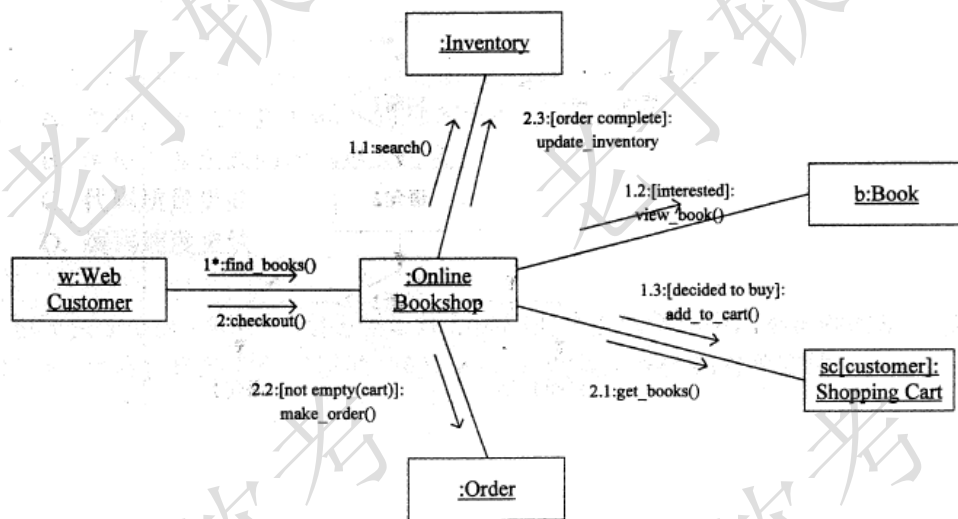
● 以下 UML 图是 (41)，图中 :Order 和 b:Book 表示 (42)，1*:find_books() 和 1.1:search() 表示 (43)。



第41题:

- A. 序列图
- B. 状态图
- C. 通信图
- D. 活动图

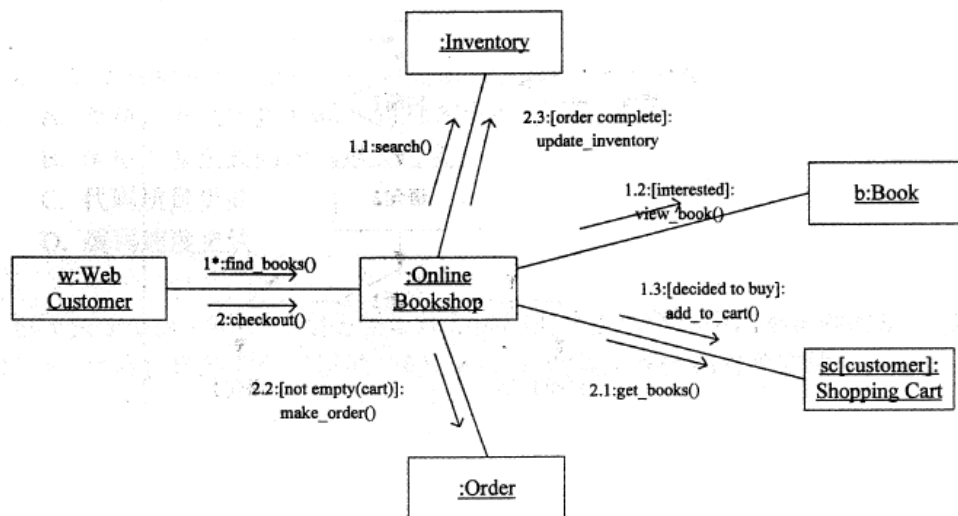
● 以下 UML 图是 (41)，图中 :Order 和 b:Book 表示 (42)，1*:find_books() 和 1.1:search() 表示 (43)。



第42题:

- A. 类
- B. 对象
- C. 流名称
- D. 消息

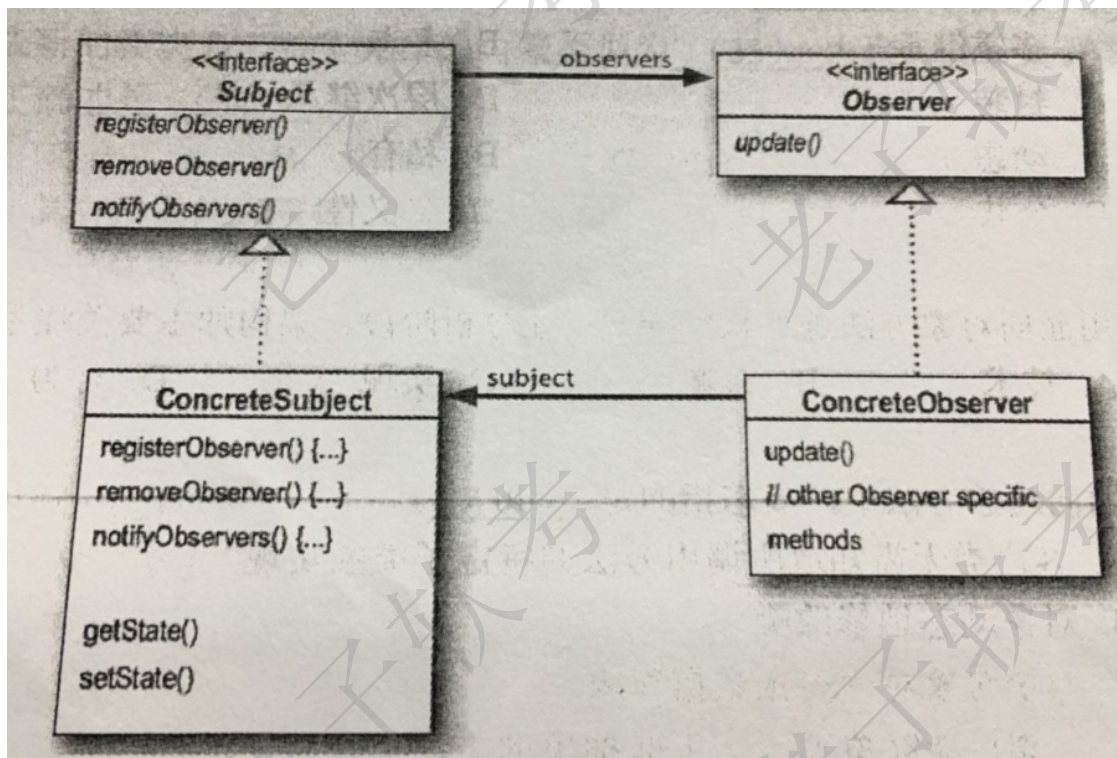
● 以下 UML 图是 (41)，图中 `:Order` 和 `b:Book` 表示 (42)，`1*:find_books()` 和 `1.1:search()` 表示 (43)。



第43题:

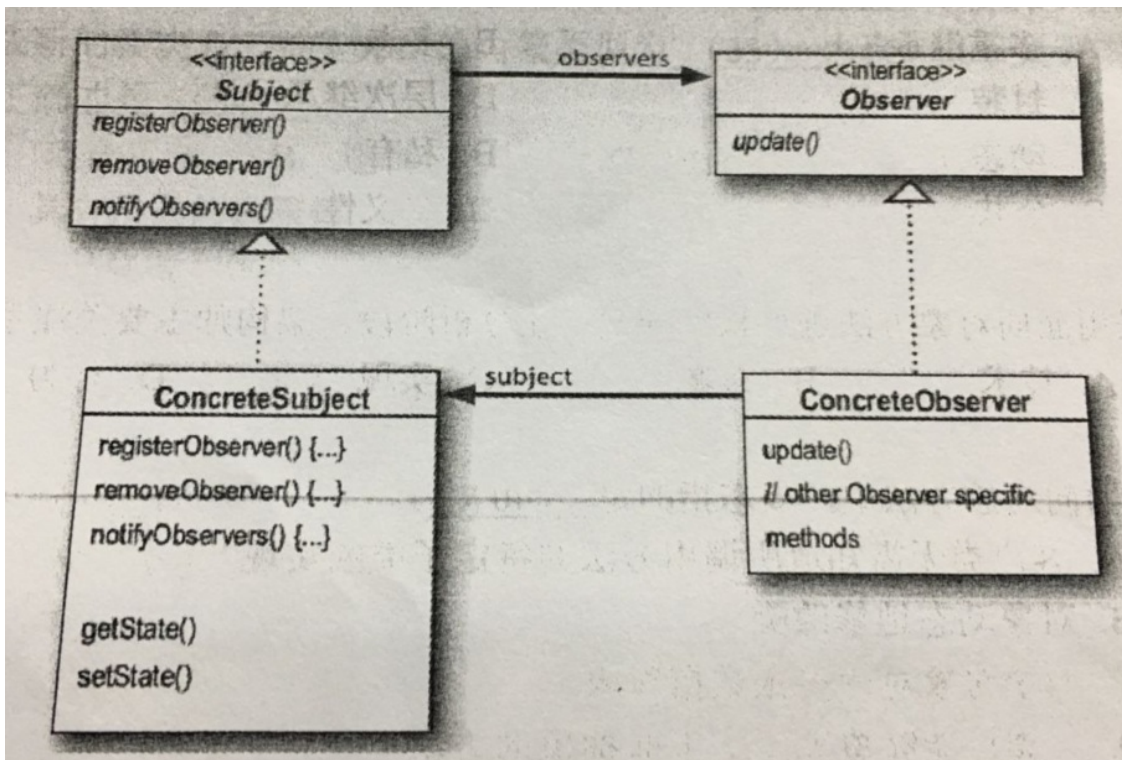
- A. 类
- B. 对象
- C. 流名称
- D. 消息

第44题: 下图所示为观察者(Observer)模式的抽象示意图, 其中()知道其观察者, 可以有任何多个观察者观察同一个目标;提供住处和删除观察者对象的接口。此模式体现的最主要的特征是()



- A. Subject
- B. Observer
- C. ConcreteSubject
- D. ConcreteObserver

第45题: 下图所示为观察者(Observer)模式的抽象示意图, 其中()知道其观察者, 可以有任何多个观察者观察同一个目标;提供住处和删除观察者对象的接口。此模式体现的最主要的特征是()



- A. 类应该对扩展开放，对修改关闭
- B. 使所要交互的对象尽量松耦合
- C. 组合优先于继承使用
- D. 仅与直接关联类交互

第46题：装饰器 (Decorator) 模式用于 ();外观 (Facade) 模式用于()。

- ①将一个对象加以包装以给客户提供其希望的另外一个接口
- ②将一个对象加以包装以提供一些额外的行为
- ③将一个对象加以包装以控制对这个对象的访问
- ④将一系列对象加以包装以简化其接口

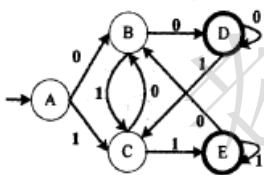
- A. ①
- B. ②
- C. ③
- D. ④

第47题：装饰器 (Decorator) 模式用于 ();外观 (Facade) 模式用于()。

- ①将一个对象加以包装以给客户提供其希望的另外一个接口
- ②将一个对象加以包装以提供一些额外的行为
- ③将一个对象加以包装以控制对这个对象的访问
- ④将一系列对象加以包装以简化其接口

- A. ①
- B. ②
- C. ③
- D. ④

第48题：某确定的有限自动机 (DFA) 的状态转换图如下图所示 (A 是初态，D、E 是终态)，则该 DFA 能识别 ()



- A. 00110
- B. 10101
- C. 11100
- D. 11001

第49题：函数main()、f()的定义如下所示，调用函数们f()时，第一个参数采用传值 (call by value) 方式，第二个参数采用传引用 (call by reference) 方式， main() 函数中 "print(x)" 执行后输出的值为 ()

main()

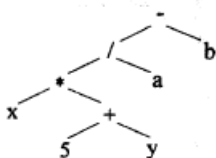
```
int x = 5;  
f(x+1, x);  
print(x);
```

f(int x, int &a)

```
x = x*x - 1;  
a = x + a;  
return;
```

- A. 11
- B. 40
- C. 45
- D. 70

第50题：下图为一个表达式的语法树，该表达式的后缀形式为()



- A. x 5 y + * a / b -
- B. x 5 y a b * + / -
- C. - / * x + 5 y a b
- D. x 5 * y + a / b -

第51题：若事务T₁对数据D₁加了共享锁，事务T₂、T₃分别对数据D₂、D₃加了排它锁，则事务T₁对数据()；事务T₂对数据()

- A. D₂、D₃加排它锁都成功
- B. D₂、D₃加共享锁都成功
- C. D₂加共享锁成功，D₃加排它锁失败
- D. D₂、D₃加排它锁和共享锁都失败

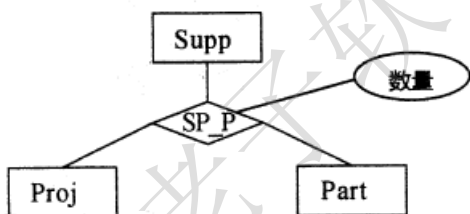
第52题：若事务T₁对数据D₁加了共享锁，事务T₂、T₃分别对数据D₂、D₃加了排它锁，则事务T₁对数据()；事务T₂对数据()

- A. D₁、D₃加共享锁都失败
- B. D₁、D₃加共享锁都成功
- C. D₁加共享锁成功，D₃加排它锁失败
- D. D₁加排它锁成功，D₃加共享锁失败

第53题：假设关系R<U,F>,U= {A₁,A₂,A₃}，F= {A₁A₃→A₂,A₁A₂→A₃}，则关系R的各候选关键字中必定含有属性()。

- A. A₁
- B. A₂
- C. A₃
- D. A₂A₃

第54题：在某企业的工程项目管理系统的数据库中供应商关系Supp、项目关系Proj和零件关系Part的E-R模型和关系模式如下：



Supp (供应商号,供应商名,地址,电话)

Proj (项目号,项目名,负责人,电话)

Part (零件号,零件名)

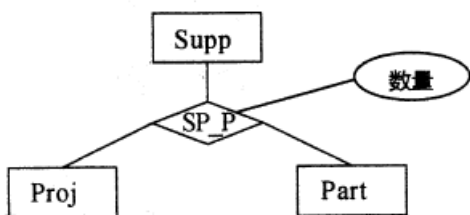
其中，每个供应商可以为多个项目供应多种零件，每个项目可由多个供应商供应多种零件。SP_P需要生成一个独立的关系模式，其联系类型为(54)

给定关系模式SP_P (供应商号,项目号,零件号,数量) 查询至少供应了3个项目 (包含3项) 的供应商，输出其供应商号和供应零件数量的总和，并按供应商号降序排列。

SELECT 供应商号, SUM (数量) FROM (55) GROUP BY 供应商号 (56) ORDER BY 供应商号 DESC;

- A. *.*.*
- B. 1:*.*
- C. 1:1:*
- D. 1:1:1

第55题：在某企业的工程项目管理系统的数据库中供应商关系Supp、项目关系Proj和零件关系Part的E-R模型和关系模式如下：



Supp（供应商号,供应商名,地址,电话）

Proj（项目号,项目名,负责人,电话）

Part（零件号,零件名）

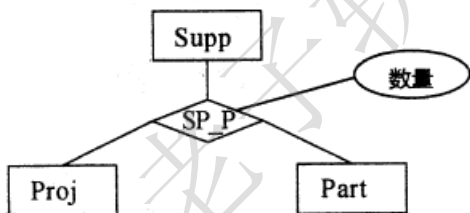
其中，每个供应商可以为多个项目供应多种零件，每个项目可由多个供应商供应多种零件。SP_P需要生成一个独立的关系模式，其联系类型为（54）

给定关系模式SP_P（供应商号,项目号,零件号,数量）查询至少供应了3个项目（包含3项）的供应商，输出其供应商号和供应零件数量的总和，并按供应商号降序排列。

SELECT 供应商号, SUM（数量） FROM (55) GROUP BY 供应商号 (56) ORDER BY 供应商号 DESC;

- A. Supp
- B. Proj
- C. Part
- D. SP_P

第56题：在某企业的工程项目管理系统的数据库中供应商关系Supp、项目关系Proj和零件关系Part的E-R模型和关系模式如下：



Supp（供应商号,供应商名,地址,电话）

Proj（项目号,项目名,负责人,电话）

Part（零件号,零件名）

其中，每个供应商可以为多个项目供应多种零件，每个项目可由多个供应商供应多种零件。SP_P需要生成一个独立的关系模式，其联系类型为（54）

给定关系模式SP_P（供应商号,项目号,零件号,数量）查询至少供应了3个项目（包含3项）的供应商，输出其供应商号和供应零件数量的总和，并按供应商号降序排列。

SELECT 供应商号, SUM（数量） FROM (55) GROUP BY 供应商号 (56) ORDER BY 供应商号 DESC;

- A. HAVING COUNT(项目号)>2
- B. WHERE COUNT(项目号)>2
- C. HAVING COUNT(DISTINCT(项目号))>2
- D. WHERE COUNT(DISTINCT(项目号))>3

第57题：以下关于字符串的叙述中，正确的是（ ）

- A. 包含任意个空格字符的字符串称为空串
- B. 字符串不是线性数据结构
- C. 字符串的长度是指串中所含字符的个数
- D. 字符串的长度是指串中所含非空格字符的个数

第58题：已知栈S 初始为空，用I表示入栈、O表示出栈，若入栈序列为 $a_1a_2a_3a_4a_5$ ，则通过栈S 得到出栈序列 $a_2a_4a_5a_3a_1$ 的合法操作序列()

- A. IIOIIOIOOO
- B. IOIOIOIOIO

- C. 1001101010
D. 1100101000

第59题：某二叉树的先序遍历序列为 ABCDEF，中序遍历序列为BADCFE，则该二叉树的高度(即层数)为()

- A. 3
B. 4
C. 5
D. 6

第60题：对于n个元素的关键字序列 $\{k_1, k_2, \dots, k_n\}$ ，当且仅当满足关系 $k_i \leq k_{2i}$ 且 $k_i \leq k_{2i+1}$ ($i=1, 2, \dots, [n/2]$) 时称其为小根堆(小顶堆)。以下序列中，()不是小根堆。

- A. 16, 25, 40, 55, 30, 50, 45
B. 16, 40, 25, 50, 45, 30, 55
C. 16, 25, 39, 41, 45, 43, 50
D. 16, 40, 25, 53, 39, 55, 45

第61题：在12个互异元素构成的有序数组 $a[1..12]$ 中进行二分查找(即折半查找，向下取整)，若待查找的元素正好等于 $a[9]$ ，则在此过程中，依次与数组中的()比较后，查找成功结束。

- A. $a[6]$ 、 $a[7]$ 、 $a[8]$ 、 $a[9]$
B. $a[6]$ 、 $a[9]$
C. $a[6]$ 、 $a[7]$ 、 $a[9]$
D. $a[6]$ 、 $a[8]$ 、 $a[9]$

第62题：某汽车加工工厂有两条装配线L1和L2，每条装配线的工位数为n (S_{ij} , $i=1$ 或2, $j=1, 2, \dots, n$)，两条装配线对应的工位完成同样的加工工作，但是所需要的时间可能不同 (a_{ij} , $i=1$ 或2, $j=1, 2, \dots, n$)。汽车底盘开始到进入两条装配线的时间 (e_1, e_2) 以及装配后到结束的时间(X_1, X_2)也可能不相同。从一个工位加工后流到下一个工位需要迁移时间(t_{ij} , $i=1$ 或2, $j=2, \dots, n$)。现在要以最快的时间完成一辆汽车的装配，求最优的装配路线。

分析该问题，发现问题具有最优子结构。以 L1 为例，除了第一个工位之外，经过第j个工位的最短时间包含了经过L1的第j-1个工位的最短时间或者经过L2的第j-1个工位的最短时间，如式(1)。装配后到结束的最短时间包含离开L1的最短时间或者离开L2的最短时间如式 (2)。

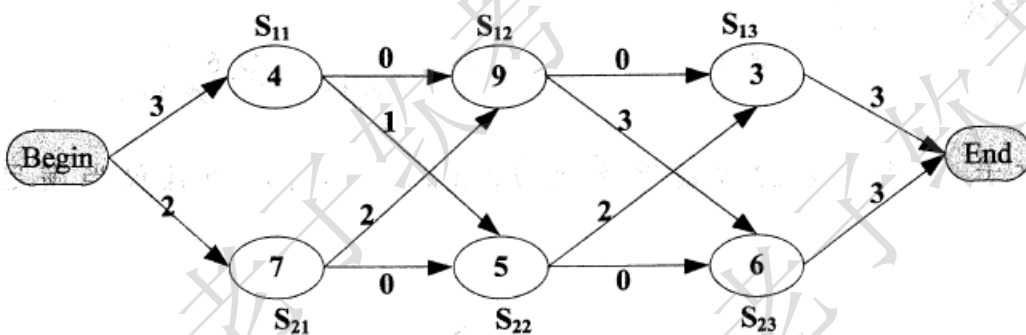
$$f_{1,j} = \begin{cases} e_1 + a_{1,j} & \text{若 } j=1 \\ \min(f_{1,j-1} + a_{1,j} + t_{1,j-1}, f_{2,j-1} + a_{1,j} + t_{2,j-1}) & \text{其他} \end{cases} \quad (1)$$

$$f_{\min} = \min(f_{1,n} + x_1, f_{2,n} + x_2) \quad (2)$$

由于在求解经过L1和L2的第j个工位的最短时间均包含了经过L1的第j-1个工位的最短时间或者经过L2的第j-1个工位的最短时间，该问题具有重复子问题的性质，故采用迭代方法求解。

该问题采用的算法设计策略是 ()，算法的时间复杂度为 ()

以下是一个装配调度实例，其最短的装配时间为 ()，装配路线为 ()



- A. 分治
B. 动态规划
C. 贪心
D. 回溯

第63题：某汽车加工工厂有两条装配线L1和L2，每条装配线的工位数为n (S_{ij} , $i=1$ 或2, $j=1, 2, \dots, n$)，两条装配线对应的工位完成同样的加工工作，但是所需要的时间可能不同 (a_{ij} , $i=1$ 或2, $j=1, 2, \dots, n$)。汽车底盘开始到进入两条装配线

的时间 (e_1, e_2) 以及装配后到结束的时间(X_1, X_2)也可能不相同。从一个工位加工后流到下一个工位需要迁移时间($t_{ij}, i=1$ 或 $2, j=2, \dots, n$)。现在要以最快的时间完成一辆汽车的装配, 求最优的装配路线。

分析该问题, 发现问题具有最优子结构。以 L_1 为例, 除了第一个工位之外, 经过第 j 个工位的最短时间包含了经过 L_1 的第 $j-1$ 个工位的最短时间或者经过 L_2 的第 $j-1$ 个工位的最短时间, 如式(1)。装配后到结束的最短时间包含离开 L_1 的最短时间或者离开 L_2 的最短时间如式 (2)。

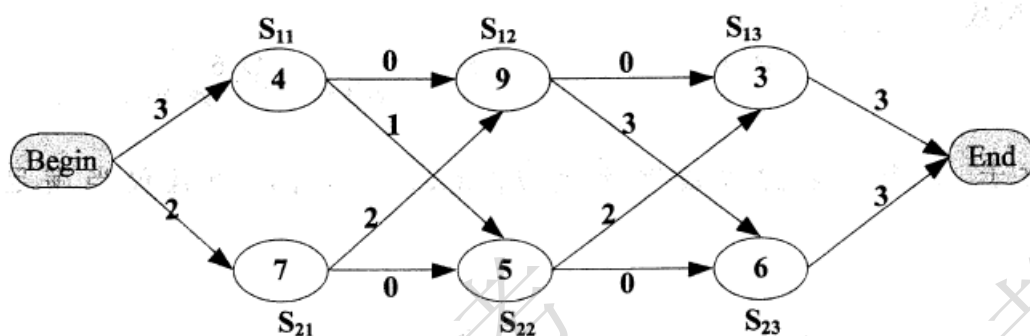
$$f_{1,j} = \begin{cases} e_1 + a_{1,j} & \text{若 } j=1 \\ \min(f_{1,j-1} + a_{1,j} + t_{1,j-1}, f_{2,j-1} + a_{1,j} + t_{2,j-1}) & \text{其他} \end{cases} \quad (1)$$

$$f_{\min} = \min(f_{1,n} + x_1, f_{2,n} + x_2) \quad (2)$$

由于在求解经过 L_1 和 L_2 的第 j 个工位的最短时间均包含了经过 L_1 的第 $j-1$ 个工位的最短时间或者经过 L_2 的第 $j-1$ 个工位的最短时间, 该问题具有重复子问题的性质, 故采用迭代方法求解。

该问题采用的算法设计策略是 (), 算法的时间复杂度为 ()

以下是一个装配调度实例, 其最短的装配时间为 (), 装配路线为 ()



- A. $O(\lg n)$
- B. $O(n)$
- C. $O(n^2)$
- D. $O(n \lg n)$

第64题: 某汽车加工工厂有两条装配线 L_1 和 L_2 , 每条装配线的工位数为 n ($S_{ij}, i=1$ 或 $2, j=1, 2, \dots, n$), 两条装配线对应的工位完成同样的加工工作, 但是所需要的时间可能不同 ($a_{ij}, i=1$ 或 $2, j=1, 2, \dots, n$)。汽车底盘开始到进入两条装配线的时间 (e_1, e_2) 以及装配后到结束的时间(X_1, X_2)也可能不相同。从一个工位加工后流到下一个工位需要迁移时间($t_{ij}, i=1$ 或 $2, j=2, \dots, n$)。现在要以最快的时间完成一辆汽车的装配, 求最优的装配路线。

分析该问题, 发现问题具有最优子结构。以 L_1 为例, 除了第一个工位之外, 经过第 j 个工位的最短时间包含了经过 L_1 的第 $j-1$ 个工位的最短时间或者经过 L_2 的第 $j-1$ 个工位的最短时间, 如式(1)。装配后到结束的最短时间包含离开 L_1 的最短时间或者离开 L_2 的最短时间如式 (2)。

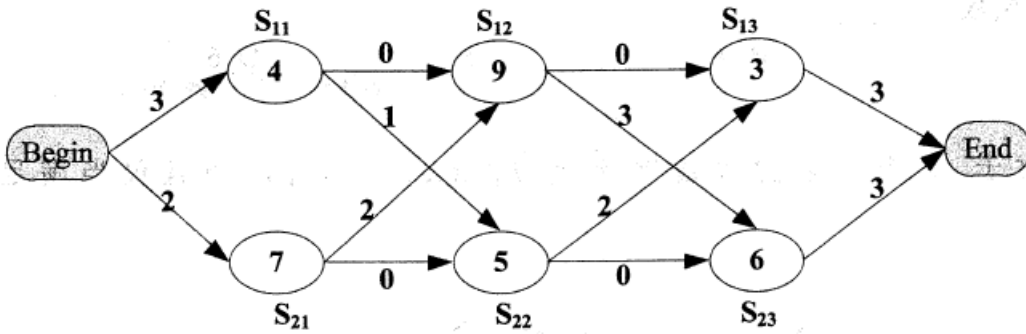
$$f_{1,j} = \begin{cases} e_1 + a_{1,j} & \text{若 } j=1 \\ \min(f_{1,j-1} + a_{1,j} + t_{1,j-1}, f_{2,j-1} + a_{1,j} + t_{2,j-1}) & \text{其他} \end{cases} \quad (1)$$

$$f_{\min} = \min(f_{1,n} + x_1, f_{2,n} + x_2) \quad (2)$$

由于在求解经过 L_1 和 L_2 的第 j 个工位的最短时间均包含了经过 L_1 的第 $j-1$ 个工位的最短时间或者经过 L_2 的第 $j-1$ 个工位的最短时间, 该问题具有重复子问题的性质, 故采用迭代方法求解。

该问题采用的算法设计策略是 (), 算法的时间复杂度为 ()

以下是一个装配调度实例, 其最短的装配时间为 (), 装配路线为 ()



- A. 21
- B. 23
- C. 20
- D. 26

第65题：某汽车加工工厂有两条装配线L1和L2，每条装配线的工位数为n (S_{ij} , $i=1$ 或 2 , $j=1, 2, \dots, n$)，两条装配线对应的工位完成同样的加工工作，但是所需要的时间可能不同 (a_{ij} , $i=1$ 或 2 , $j=1, 2, \dots, n$)。汽车底盘开始到进入两条装配线的时间 (e_1, e_2) 以及装配后到结束的时间(x_1, x_2)也可能不相同。从一个工位加工后流到下一个工位需要迁移时间(t_{ij} , $i=1$ 或 2 , $j=2, \dots, n$)。现在要以最快的时间完成一辆汽车的装配，求最优的装配路线。

分析该问题，发现问题具有最优子结构。以 L1 为例，除了第一个工位之外，经过第j个工位的最短时间包含了经过L1的第j-1个工位的最短时间或者经过L2的第j-1个工位的最短时间，如式(1)。装配后到结束的最短时间包含离开L1的最短时间或者离开L2的最短时间如式 (2)。

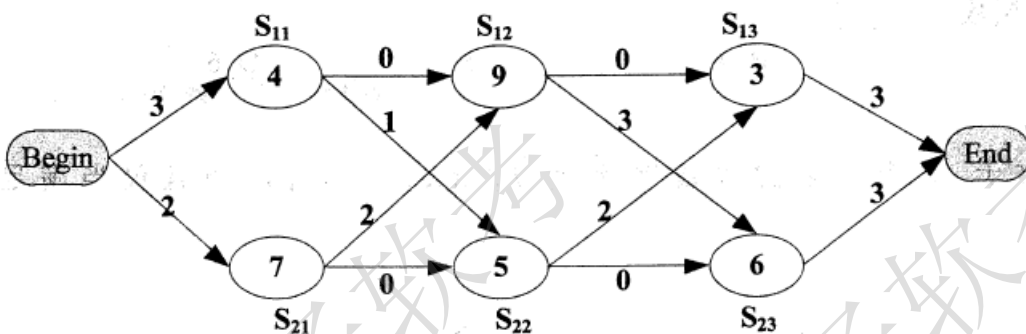
$$f_{1,j} = \begin{cases} e_1 + a_{1,j} & \text{若 } j=1 \\ \min(f_{1,j-1} + a_{1,j} + t_{1,j-1}, f_{2,j-1} + a_{1,j} + t_{2,j-1}) & \text{其他} \end{cases} \quad (1)$$

$$f_{\min} = \min(f_{1,n} + x_1, f_{2,n} + x_2) \quad (2)$$

由于在求解经过L1和L2的第j个工位的最短时间均包含了经过L1的第j-1个工位的最短时间或者经过L2的第j-1个工位的最短时间，该问题具有重复子问题的性质，故采用迭代方法求解。

该问题采用的算法设计策略是 ()，算法的时间复杂度为 ()

以下是一个装配调度实例，其最短的装配时间为 ()，装配路线为 ()



- A. S11→S12→S13
- B. S11→S22→S13
- C. S21→S12→S23
- D. S21→S22→S23

第66题：在浏览器地址栏输入一个正确的网址后，本地主机将首先在()查询该网址对应的IP地址。

- A. 本地DNS缓存
- B. 本机hosts文件
- C. 本地DNS服务器
- D. 根域名服务器

第67题：下面关于Linux目录的描述中，正确的是()

- A. Linux只有一个根目录，用"/root"表示
- B. Linux中有多个根目录，用"/"加相应目录名称表示
- C. Linux中只有一个根目录，用"/"表示
- D. Linux中有多个根目录，用相应目录名称表示

第68题：以下关于TCP/IP 协议栈中协议和层次的对应关系正确的是（）

A.

TFTP	Telnet
UDP	TCP
ARP	

B.

RIP	Telnet
UDP	TCP
ARP	

C.

HTTP	SNMP
TCP	UDP
IP	

D.

SMTP	FTP
UDP	TCP
IP	

第69题：在异步通信中，每个字符包含 1 位起始位、7位数据位和2位终止位，若每秒钟传送500个字符，则有效数据速率为()

- A. 500b/s
- B. 700b/s
- C. 3500b/s
- D. 5000b/s

第70题：以下路由策略中，依据网络信息经常更新路由的是()

- A. 静态路由
- B. 洪泛式
- C. 随机路由
- D. 自适应路由

第71题：The beauty of software is in its function, in its internal structure, and in the way in which it is created by a team. To a user, a program with just the right features presented through an intuitive and (71)interface is beautiful. To a software designer, an internal structure that is partitioned in a simple and intuitive manner, and that minimizes internal coupling is beautiful. To developers and managers, a motivated team of developers making significant progress every week, and producing defect-free code, is beautiful. There is beauty on all these levels.

our world needs software--lots of software. Fifty years ago software was something that ran in a few big and expensive machines. Thirty years ago it was something that ran in most companies and industrial settings. Now there is software running in our cell phones, watches, appliances, automobiles, toys, and tools. And need for new and better software never(72). As our civilization grows and expands, as developing nations build their infrastructures, as developed nations strive to achieve ever greater efficiencies, the need for more and more Software(73)to increase. It would be a great shame if, in all that software, there was no beauty.

We know that software can be ugly. We know that it can be hard to use, unreliable, and carelessly structured. We know that there are software systems whose tangled and careless internal structures make them expensive and difficult to change. We know that there are software systems that present their features through an awkward and cumbersome interface. We know that there are software systems that crash and misbehave. These are(74) systems. Unfortunately, as a profession, software developers tend to create more ugly systems than beautiful ones. There is a secret that the best software developers know. Beauty is cheaper than ugliness. Beauty is faster than ugliness. A beautiful software system can be built and maintained in less time, and for less money, than an ugly one. Novice software developers don't. understand this. They think that they have to do everything fast and quick. They think that beauty is(75). No! By doing things fast and quick, they make messes that make the software stiff, and hard to understand. Beautiful systems are flexible and easy to understand. Building them and maintaining them is a joy. It is ugliness that is impractical. Ugliness will slow you down and make your software expensive and brittle. Beautiful systems cost the least build and maintain, and are delivered soonest.

- A. Simple
- B. Hard
- C. Complex
- D. duplicated

第72题：The beauty of software is in its function, in its internal structure, and in the way in which it is created by a team. To a user, a program with just the right features presented through an intuitive and (71)interface is beautiful. To a software designer, an internal structure that is partitioned in a simple and intuitive manner, and that minimizes internal coupling is beautiful. To developers and managers, a motivated team of developers making significant progress every week, and producing defect-free code, is beautiful. There is beauty on all these levels. our world needs software--lots of software. Fifty years ago software was something that ran in a few big and expensive machines. Thirty years ago it was something that ran in most companies and industrial settings. Now there is software running in our cell

phones, watches, appliances, automobiles, toys, and tools. And need for new and better software never(72).As our civilization grows and expands, as developing nations build their infrastructures, as developed nations strive to achieve ever greater efficiencies, the need for more and more Software(73)to increase. It would be a great shame if, in all that software, there was no beauty.

We know that software can be ugly. We know that it can be hard to use, unreliable, and carelessly structured. We know that there are software systems whose tangled and careless internal structures make them expensive and difficult to change. We know that there are software systems that present their features through an awkward and cumbersome interface. We know that there are software systems that crash and misbehave. These are(74) systems. Unfortunately, as a profession, software developers tend to create more ugly systems than beautiful ones. There is a secret that the best software developers know. Beauty is cheaper than ugliness. Beauty is faster than ugliness. A beautiful software system can be built and maintained in less time, and for less money, than an ugly one. Novice software developers don't understand this. They think that they have to do everything fast and quick. They think that beauty is(75).No! By doing things fast and quick, they make messes that make the software stiff, and hard to understand, Beautiful systems are flexible and easy to understand. Building them and maintaining them is a joy. It is ugliness that is impractical.Ugliness will slow you down and make your software expensive and brittle. Beautiful systems cost the least build and maintain, and are delivered soonest.

- A. happens
- B. exists
- C. stops
- D. starts

第73题: The beauty of software is in its function, in its internal structure, and in the way in which it is created by a team. To a user, a program with just the right features presented through an intuitive and(71)interface is beautiful.To a software designer, an internal structure that is partitioned in a simple and intuitive manner, and that minimizes internal coupling is beautiful.To developers and managers, a motivated team of developers making significant progress every week, and producing defect-free code, is beautiful. There is beauty on all these levels.

our world needs software--lots of software. Fifty years ago software was something that ran in a few big and expensive machines. Thirty years ago it was something that ran in most companies and industrial settings. Now there is software running in our cell

phones, watches, appliances, automobiles, toys, and tools. And need for new and better software never(72).As our civilization grows and expands, as developing nations build their infrastructures, as developed nations strive to achieve ever greater efficiencies, the need for more and more Software(73)to increase. It would be a great shame if, in all that software, there was no beauty.

We know that software can be ugly. We know that it can be hard to use, unreliable, and carelessly structured. We know that there are software systems whose tangled and careless internal structures make them expensive and difficult to change. We know that there are software systems that present their features through an awkward and cumbersome interface. We know that there are software systems that crash and misbehave. These are(74) systems. Unfortunately, as a profession, software developers tend to create more ugly systems than beautiful ones. There is a secret that the best software developers know. Beauty is cheaper than ugliness. Beauty is faster than ugliness. A beautiful software system can be built and maintained in less time, and for less money, than an ugly one. Novice software developers don't understand this. They think that they have to do everything fast and quick. They think that beauty is(75).No! By doing things fast and quick, they make messes that make the software stiff, and hard to understand, Beautiful systems are flexible and easy to understand. Building them and maintaining them is a joy. It is ugliness that is impractical.Ugliness will slow you down and make your software expensive and brittle. Beautiful systems cost the least build and maintain, and are delivered soonest.

- A. starts
- B. continues
- C. appears
- D. stops

第74题: The beauty of software is in its function, in its internal structure, and in the way in which it is created by a team. To a user, a program with just the right features presented through an intuitive and(71)interface is beautiful.To a software designer, an internal structure that is partitioned in a simple and intuitive manner, and that minimizes internal coupling is beautiful.To developers and managers, a motivated team of developers making significant progress every week, and producing defect-free code, is beautiful. There is beauty on all these levels.

our world needs software--lots of software. Fifty years ago software was something that ran in a few big and expensive machines. Thirty years ago it was something that ran in most companies and industrial settings. Now there is software running in our cell

phones, watches, appliances, automobiles, toys, and tools. And need for new and better software never(72).As our civilization grows and expands, as developing nations build their infrastructures, as developed nations strive to achieve ever greater efficiencies, the need for more and more Software(73)to increase. It would be a great shame if, in all that software, there was no beauty.

We know that software can be ugly. We know that it can be hard to use, unreliable, and carelessly structured. We know that there are software systems whose tangled and careless internal structures make them expensive and difficult to change. We know that there are software systems that present their features through an awkward and cumbersome interface. We know that there are software systems that crash and misbehave. These are(74) systems. Unfortunately, as a profession, software developers tend to create more ugly systems than beautiful ones. There is a secret that the best software developers know. Beauty is cheaper than ugliness. Beauty is faster than ugliness. A beautiful software system can be built and maintained in less time, and for less money, than an ugly one. Novice software developers don't understand this. They think that they have to do everything fast and quick. They think that beauty is(75).No! By doing things fast and quick, they make messes that make the software stiff, and hard to understand, Beautiful systems are flexible and easy to understand. Building them and maintaining them is a joy. It is ugliness that is impractical.Ugliness will slow you down and make your software expensive and brittle. Beautiful systems cost the least build and maintain, and are delivered soonest.

- A. practical
- B. useful
- C. beautiful
- D. ugly

第75题: The beauty of software is in its function, in its internal structure, and in the way in which it is created by a team. To a user, a program with just the right features presented through an intuitive and (71) interface is beautiful. To a software designer, an internal structure that is partitioned in a simple and intuitive manner, and that minimizes internal coupling is beautiful. To developers and managers, a motivated team of developers making significant progress every week, and producing defect-free code, is beautiful. There is beauty on all these levels. Our world needs software--lots of software. Fifty years ago software was something that ran in a few big and expensive machines. Thirty years ago it was something that ran in most companies and industrial settings. Now there is software running in our cell phones, watches, appliances, automobiles, toys, and tools. And need for new and better software never (72). As our civilization grows and expands, as developing nations build their infrastructures, as developed nations strive to achieve ever greater efficiencies, the need for more and more Software (73) to increase. It would be a great shame if, in all that software, there was no beauty. We know that software can be ugly. We know that it can be hard to use, unreliable, and carelessly structured. We know that there are software systems whose tangled and careless internal structures make them expensive and difficult to change. We know that there are software systems that present their features through an awkward and cumbersome interface. We know that there are software systems that crash and misbehave. These are (74) systems. Unfortunately, as a profession, software developers tend to create more ugly systems than beautiful ones. There is a secret that the best software developers know. Beauty is cheaper than ugliness. Beauty is faster than ugliness. A beautiful software system can be built and maintained in less time, and for less money, than an ugly one. Novice software developers don't understand this. They think that they have to do everything fast and quick. They think that beauty is (75). No! By doing things fast and quick, they make messes that make the software stiff, and hard to understand. Beautiful systems are flexible and easy to understand. Building them and maintaining them is a joy. It is ugliness that is impractical. Ugliness will slow you down and make your software expensive and brittle. Beautiful systems cost the least to build and maintain, and are delivered soonest.

- A. impractical
- B. perfect
- C. time-wasting
- D. practical